THE HEART CENTER · INFORMATION TECHNOLOGY

Intranet Platform

Technical Documentation & Change Management Reference

Document Number	HCI-IT-CMD-001
Version	1.1
Effective Date	June 01, 2026
Classification	Internal Use Only
Owner	IT Operations
Review Cycle	Annual or upon major change
Current Deployment	Homelab — Proxmox VE (KVM)
Production Target	Huntsville Hospital VMware vSphere https://Intranet-HCI.heart.local (planned)

Document Control

Authorship & Approval

Role	Name	Signature	Date
Prepared by			
Technical Reviewer			
IT Director / Approver			
Security Reviewer			
Database Administrator			

Revision History

Version	Date	Author	Description of Change
1.1	June 01, 2026	IT Operations	Documentation realigned to match the actual deployed system. Documents the current homelab environment (Proxmox VE, RHEL 10, Express, SQLite, Tailscale, RDP) as the primary state, with explicit "Production migration" callouts at the end of every chapter describing the planned changes for the Huntsville Hospital VMware vSphere production rollout (Fastify, PostgreSQL on host, Prisma, HTTPS at Intranet-HCI.heart.local, corporate network only).
1.0	Previous	IT Operations	Initial production-target documentation set. Described the Huntsville Hospital vSphere deployment as if already implemented. Superseded by v1.1 which separates current and target states.

Distribution List

Role	Distribution Method
IT Operations Team	Internal git repository
IT Director	Email / shared drive
Database Administrators	Hospital shared drive
Designated Backup Operator	Printed copy in operations binder

Related Documents

Reference	Title
HCI-IT-CMD-001-A	Huntsville Hospital Information Security Policy
HCI-IT-CMD-001-B	Incident Response Plan
HCI-IT-CMD-001-C	Acceptable Use Policy
HCI-IT-CMD-001-D	Change Management Procedure
HCI-IT-CMD-001-E	PostgreSQL Administration Standard

Purpose and Scope

Purpose. This document provides the canonical technical reference for The Heart Center employee intranet platform. It documents both the current homelab test deployment and the planned Huntsville Hospital production environment. It defines the architecture, documents operational procedures, codifies recovery actions, and serves as the change-management baseline for the system.

Scope. Covers two states of the same system: (1) the *current homelab deployment* running on Proxmox VE with an Express backend and SQLite database, and (2) the *planned Huntsville Hospital production deployment* on VMware vSphere with a Fastify backend and PostgreSQL database. The body of each chapter documents the current state; a *Production migration* callout at the end of each chapter lists the changes planned for production cutover. Does not cover the Huntsville Hospital network infrastructure, the vSphere cluster management, or PostgreSQL server-level administration beyond the scope of this single database.

Intended audience. IT Operations personnel, the IT Director, the Security Reviewer, Database Administrators, and designated backup operators.

Change control. Any modification to the production system that invalidates a procedure in this document constitutes a change requiring an updated revision of this document. Update the Revision History table when this occurs. Schema changes in the current homelab Express deployment are inline in server.js; in production they will be Prisma migrations committed to source control.

1 01 — System Overview

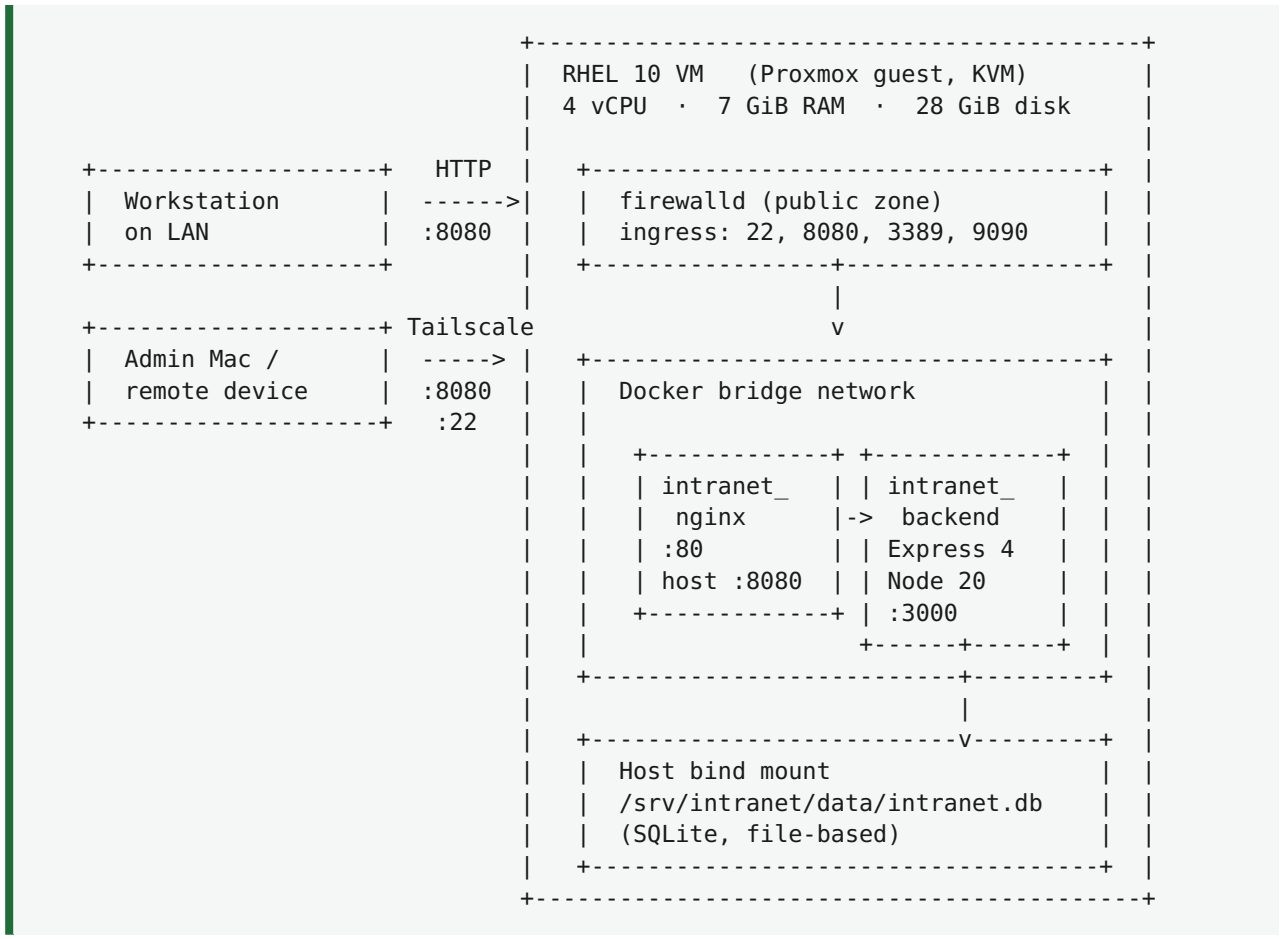
1.1 What this system is

The **Heart Center Intranet** is an internal-only web platform for The Heart Center medical practice. It provides employees with:

- Company announcements and news posts (with administrative moderation)
- Employee directory and search
- HR resources, policies, IT support requests
- Shared schedules, sign-up sheets, and event coordination
- Recognition programs (Employee Spotlight nominations)
- Centralized links to web applications used by the practice

1.2 Current deployment (homelab)

The system today runs in a **homelab test environment** as a single Red Hat Enterprise Linux 10 virtual machine on a Proxmox hypervisor. Remote access is via Tailscale or the local LAN. The site is reached at `http://<vm-ip>:8080/` over plain HTTP.


RHEL 10 VM (Proxmox guest, KVM)
4 vCPU · 7 GiB RAM · 28 GiB disk
Workstation on LAN
HTTP :8080
Admin Mac / remote device
Tailscale :8080 :22
firewalld (public zone)
ingress: 22, 8080, 3389, 9090
Docker bridge network
intranet_nginx
:80
host :8080
intranet_backend
Express 4
Node 20
:3000
Host bind mount
/srv/intranet/data/intranet.db
(SQLite, file-based)

1.3 Service layers (current)

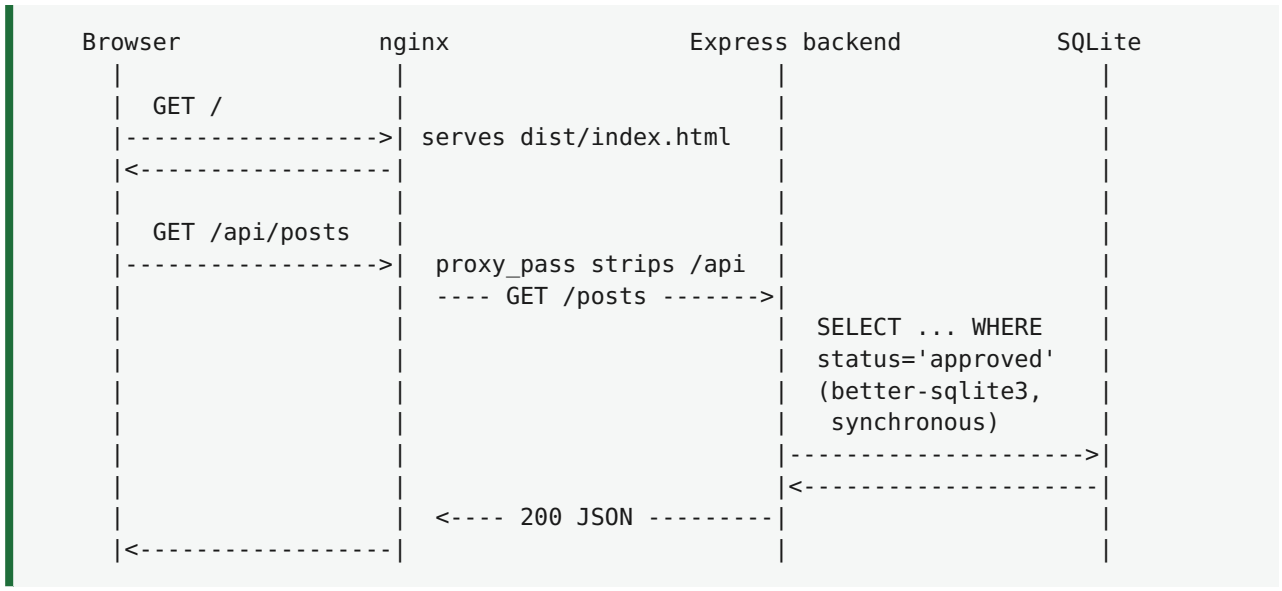
```
+-------------------------------------------------------------+
|                  (5)  USER EXPERIENCE                       |
|  React 19 SPA  *  Admin HTML portal  *  Manager HTML portal |
+-------------------------------------------------------------+
|                  (4)  APPLICATION                           |
|  Express 4 backend (Node.js 20)  *  express-session         |
|  better-sqlite3  *  4 role tiers  *  audit logging          |
+-------------------------------------------------------------+
|                  (3)  DATA                                  |
|  SQLite databases (intranet.db, sessions.db)                |
|  Bind-mounted from host /srv/intranet/data/                 |
|  Uploaded files in /srv/intranet/uploads/                   |
+-------------------------------------------------------------+
|                  (2)  PLATFORM                              |
|  nginx reverse proxy (HTTP)  *  Docker  *  bind-mount volumes|
+-------------------------------------------------------------+
|                  (1)  INFRASTRUCTURE                        |
|  RHEL 10  *  Proxmox VE  *  firewalld  *  Tailscale  *  GDM |
+-------------------------------------------------------------+
```

1.4 Audiences served (current)

Audience	How they reach the system	Where they land
Employees on LAN	`http://<lan-ip>:8080/`	React SPA (employee view)
Managers	`http://<lan-ip>:8080/manager/`	Static manager portal
Admins / Super-admins	`http://<lan-ip>:8080/admin/`	Static admin portal
Remote operator (over Tailscale)	SSH or RDP to `<tailscale-ip>`	Linux shell or GNOME desktop
System maintenance	`https://<vm-ip>:9090/`	Cockpit web console

1.5 Data-flow at a glance

A typical “employee views the home page” request:



1.6 🔄 Production migration

When this system moves from the homelab to the Huntsville Hospital production environment, the following components change. The application code, frontend, nginx configuration topology, and overall architecture stay the same; only the specific implementations of each layer change.

Layer	Homelab (today)	Production target
Hypervisor	Proxmox VE (KVM)	VMware vSphere / ESXi (Huntsville cluster)
Network access	Tailscale + LAN	Corporate network only
Web entrypoint	`http://<ip>:8080`	`https://Intranet-HCI.heart.local`
TLS	None (plain HTTP)	Internal-CA-issued cert in nginx, port 443
Backend framework	Express 4 (`server.js`)	Fastify (modular `src/`)
Backend dependencies	`express` , `express-session` , `better-sqlite3`	`fastify` , `@fastify/session` , `@prisma/client`
Database	SQLite (file in `data/`)	PostgreSQL 16 (native systemd service on host)
Schema management	Inline `db.exec()` in `server.js`	Prisma migrations under `prisma/migrations/`
Session store	SQLite via `connect-sqlite3`	PostgreSQL via `@fastify/session`
DBA query path	`docker exec` + `sqlite3`	`psql` / DBeaver / pgAdmin to host:5432
Remote admin GUI	RDP + Cockpit	Cockpit only

The migration plan tracks these changes in `08-disaster-recovery.md` under the “Migration plan” section. Until the migration is complete, this documentation describes both states.

Continue to `02-infrastructure.md` for the server-side details.

2 02 — Infrastructure

This document describes the **server, operating system, network, and supporting services** that host the intranet in the homelab test environment today, and the changes planned for the Huntsville Hospital production deployment.

2.1 Host inventory (current homelab)

Property	Value
Operating system	Red Hat Enterprise Linux 10.1 (Coughlan)
Kernel	`6.12.0-124.8.1.el10_1.x86_64`
Virtualization	KVM guest under Proxmox VE
Proxmox host name	(homelab)
VMID on Proxmox	101
Hostname	`localhost` (transient)
Time zone	`America/Chicago`
vCPU	4
vRAM	7 GiB
vDisk	28 GiB single LVM volume `/dev/mapper/rhel-root`
Subscription	Red Hat Developer subscription, registered
Tailscale	Active, joins the `<owner>@` tailnet

To check current numbers at any time:

```
$ hostnamectl              # OS and machine identity
$ free -h                  # memory
$ df -h /                  # disk
$ uptime                   # load and uptime
```

2.2 Disk layout

```
/dev/mapper/rhel-root  <-- single LVM logical volume
    |
    +- mounted at /
           |
           +- /home/<ops-user>/         (admin user's home directory)
           |     +- .claude/            (Claude Code state)
           |
           +- /srv/intranet/            (APPLICATION - this project)
           |     +- app/                (Express backend source + node_modules)
           |     +- frontend/           (React source + built dist/)
           |     +- nginx/              (reverse-proxy config)
           |     +- public/             (admin/ and manager/ static portals)
           |     +- data/               (SQLite databases - live)
           |     +- uploads/            (uploaded files)
           |     +- docs/               (this documentation)
           |
           +- /var/lib/docker/          (Docker images and overlay storage)
           +- /var/log/                 (System logs - journald)
           +- /etc/                     (System configuration)
```

In the homelab, the SQLite databases live at `/srv/intranet/data/` and are bind-mounted into the backend container at `/data/`. There is no PostgreSQL installation yet.

2.3 Network topology (homelab)

The VM has **three logical network presences** in the homelab:

```
+-----------------------------------------------------------+
|                  RHEL 10 VM                               |
|                                                           |
|  lo            127.0.0.1/8        <-- localhost only      |
|                                                           |
|  ens18         <lan-ip>/24                                |
|                Homelab LAN                                |
|                                                           |
|  tailscale0    <tailscale-ip>/32                          |
|                Tailscale overlay                          |
|                                                           |
|  docker0       172.18.0.1/16      <-- Docker bridge       |
|                (container-to-container only)              |
+-----------------------------------------------------------+
```

Interface	Purpose	Reachable from
`lo`	Loopback	the VM itself
`ens18`	Primary NIC on the homelab LAN	LAN devices
`tailscale0`	Encrypted overlay for remote access	Authorized devices on the tailnet
`docker0`	Inter-container traffic	Containers only

2.4 Firewall (firewalld)

`firewalld` is active using the default `public` zone, bound to `ens18`. Open ingress ports today:

Port	Protocol	Service	Allowed from
`22/tcp`	TCP	OpenSSH	LAN + Tailscale
`8080/tcp`	TCP	nginx (HTTP)	LAN + Tailscale
`3389/tcp`	TCP	RDP (gnome-remote-desktop)	LAN + Tailscale
`9090/tcp`	TCP	Cockpit web console	LAN + Tailscale
`546/udp`	UDP	DHCPv6 client	LAN
`icmp`	—	ping / discovery	LAN

To inspect or modify:

```
$ sudo firewall-cmd --list-all                          # show current
$ sudo firewall-cmd --permanent --add-port=N/tcp        # open a port persistently
$ sudo firewall-cmd --reload                            # apply changes
```

2.5 Services running on the host

```
   systemd --+-- docker.service              <-- runs the intranet containers
           +-- firewalld.service           <-- packet filtering
           +-- gdm.service                 <-- GNOME display manager (for RDP)
           +-- gnome-remote-desktop.service <-- RDP server (headless)
           +-- sshd.service                <-- SSH
           +-- tailscaled.service          <-- Tailscale agent
           +-- cockpit.socket              <-- on-demand Cockpit
           +-- rhsm.service / rhsmcertd    <-- Red Hat subscription
```

To list all running services:

```
$ systemctl list-units --type=service --state=running
```

2.6 Listening sockets (current state)

```
   PORT    BIND ADDRESS                 SERVICE
   22      0.0.0.0  +  [::]             sshd
   631     127.0.0.1  +  [::1]          CUPS (local printing only)
   3389    *                            gnome-remote-desktop (RDP)
   8080    0.0.0.0  +  [::]             nginx (HTTP, served by container)
   9090    *                            cockpit
```

To check anytime:

```
$ sudo ss -tlnp
```

2.7 Boot-time service order

```
   BIOS  ->  GRUB  ->  kernel  ->  systemd  --+-->  network.target
                                              +-->  firewalld
                                              +-->  tailscaled       (joins tailnet)
                                              +-->  sshd
                                              +-->  docker           (starts
containers)
                                              +-->  graphical.target
                                              |       +-->  gdm
                                              +-->  gnome-remote-desktop (RDP ready)
```

2.8 Cockpit (web admin)

Cockpit lets you do most maintenance through a browser:

```
   https://<tailscale-ip>:9090/   ← via Tailscale
   https://<lan-ip>:9090/         ← via LAN
```

Accept the self-signed cert warning, then log in with your local user. Cockpit is the friendliest path for non-CLI maintenance.

2.9 Updates and subscription

```
$ sudo subscription-manager status                    # registered?
$ sudo dnf check-update                               # what's available
$ sudo dnf upgrade                                    # apply (asks y/n)
$ sudo dnf needs-restarting -r                        # reboot needed?
```

2.9.1 Recommended update cadence (current)

Cadence	Action
Weekly	`sudo dnf check-update`
Monthly	`sudo dnf upgrade --security`
Quarterly	Full `sudo dnf upgrade` + reboot

2.10 🔄 Production migration

For the Huntsville Hospital deployment, the infrastructure layer changes significantly:

Item	Homelab (today)	Production target
Hypervisor	Proxmox VE	VMware vSphere / ESXi
VM disk size	28 GiB	80 GiB (room for PostgreSQL + uploads)
vRAM	7 GiB	8 GiB
Hostname	`localhost` (transient)	`Intranet-HCI` (DNS A record)
FQDN	(none)	`Intranet-HCI.heart.local`
Network	Tailscale + LAN	Hospital corporate VLAN only
Firewall ports	22, 8080, 3389, 9090	22 (jump host), 443, 5432 (DBAs), 9090
TLS	(none)	Hospital internal CA cert
Remote GUI	RDP via gnome-remote-desktop	(removed — Cockpit only)
Tailscale	Active	Not installed
New service	(none)	PostgreSQL 16 systemd service

The Huntsville Hospital network team will assign the static IP and DNS name. Hospital PKI will issue the TLS certificate. Hospital security may require additional source-IP restrictions on ports 22 and 9090.

2.11 Where to go next

- `03-application.md` — the intranet code itself
- `04-deployment.md` — Docker Compose mechanics
- `05-remote-access.md` — SSH, RDP, Tailscale, Cockpit

3 03 — Application Architecture

This document describes **what the intranet software actually is today**, how its pieces talk to one another, and how data flows through it. The application's current implementation is Express + SQLite; the planned production implementation is Fastify + PostgreSQL + Prisma. Both are documented; the "Production migration" callout at the end of this chapter spells out the mapping.

3.1 Codebase layout (current)

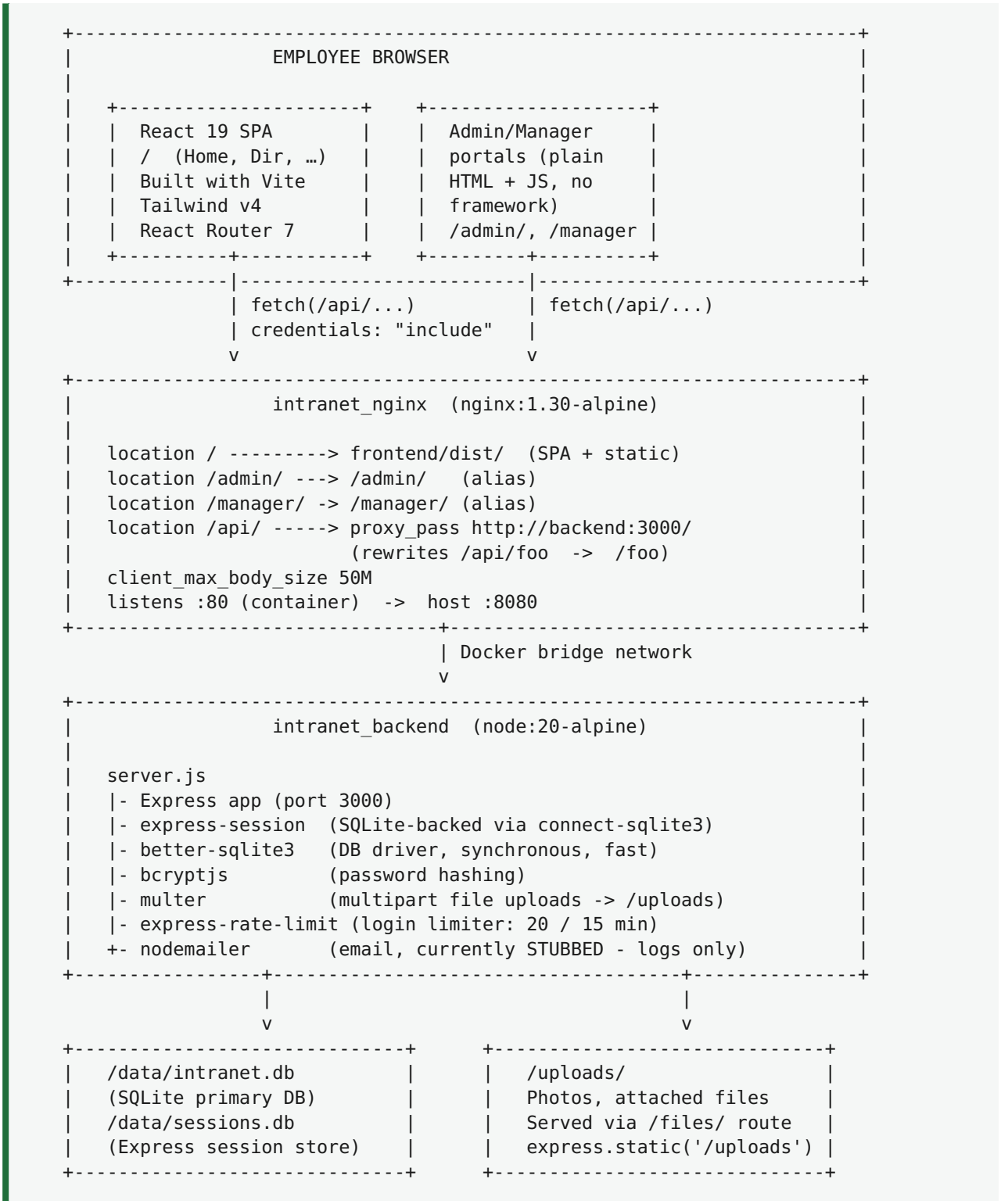
```
/srv/intranet/
├── docker-compose.yml          (two-service definition)
├── .env                        (SESSION_SECRET — sensitive)
│
├── app/                        ◄ BACKEND
│   ├── server.js               (≈1,244 lines, monolithic Express app)
│   ├── package.json            (dependencies — see below)
│   └── node_modules/           (installed on container boot)
│
├── frontend/                   ◄ EMPLOYEE-FACING SPA
│   ├── src/                    (React 19 source)
│   │   ├── main.jsx
│   │   ├── App.jsx             (routes)
│   │   ├── layouts/            (MainLayout)
│   │   ├── pages/              (Home, Announcements, Directory, …)
│   │   ├── components/         (AnnouncementCard, EmployeeSpotlight)
│   │   ├── context/            (SettingsContext)
│   │   ├── services/api.js     (centralized fetch wrapper)
│   │   └── hooks/, utils/
│   ├── dist/                   ◄ PRODUCTION BUILD (served by nginx)
│   ├── vite.config.js
│   └── package.json
│
├── public/                     ◄ STATIC PORTALS (no build step)
│   ├── admin/  (login.html, dashboard.html)
│   └── manager/ (index.html)
│
├── nginx/
│   └── default.conf            (reverse-proxy rules)
│
├── data/                       ◄ PERSISTENT SQLite DATA
│   ├── intranet.db             (primary database, ~70 KB current size)
│   └── sessions.db             (Express session store, ~12 KB)
│
├── uploads/                    ◄ UPLOADED FILES (photos, resources)
│
└── docs/                       (you are here)
```

3.2 Backend dependencies (current)

From `app/package.json`:

```
{
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "better-sqlite3": "^11.8.1",
    "connect-sqlite3": "^0.9.16",
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "express-session": "^1.18.1",
    "multer": "^1.4.5-lts.1",
    "express-rate-limit": "^7.5.0",
    "nodemailer": "^8.0.7"
  }
}
```

3.3 Component map (current)



Because nginx **strips the `/api` prefix** before proxying, every route defined in `server.js` is written **without** the `/api/` prefix. For example, `app.get('/posts')` is what the browser reaches at `/api/posts`. Keep this in mind when adding new endpoints.

3.4 Database schema (current — SQLite)

The schema is defined inline at the top of `server.js` via `db.exec()` and runs every time the backend starts. There are **no migration files** in the current implementation.

```
intranet.db (SQLite)
|- users                       (login accounts, roles)
|- posts                       (announcements with moderation workflow)
|- resources                   (HR / Policy / etc. file listings)
|- schedules                   (department schedules)
|- spotlight                   (Employee Spotlight history)
|- spotlight_nominations       (incoming nominations awaiting review)
|- directory                   (employee directory entries)
|- audit_log                   (super-admin actions for traceability)
|- site_settings               (site-wide configuration: key/value)
|- signup_sheets               (events open for signup)
|- signup_entries              (one row per person signed up)
+- it_tickets                  (IT support requests)

sessions.db (SQLite, separate file)
+- sessions                    (active Express sessions)
```

To inspect any table (current SQLite version):

```
$ docker compose exec backend sh
/app # apk add --no-cache sqlite     # one-time per container life
/app # sqlite3 /data/intranet.db
sqlite> .tables
sqlite> .schema users
sqlite> SELECT id, username, role FROM users;
sqlite> .quit
```

3.5 Roles and authorization

Four authorization tiers, implemented as middleware in `server.js`:

```
                              +----------------------------------+
                              |  Public (no login)               |
                              |  • Home, posts list, directory   |
                              +----------------+-----------------+
                                               | login
                                               v
                              +----------------------------------+
                              |  requireLogin                    |
                              |  any authenticated user          |
                              +----------------+-----------------+
                                               | role check
            +-----------------+-----------------+-----------------+
            v                 v                 v                 v
  +------------------+ +----------------+ +----------------+ +--------------+
  | requireAdmin     | | requireApprover| | requireSuper-  | |              |
  | manager+admin    | | admin or       | | Admin          | |              |
  | +superadmin      | | superadmin     | | superadmin only| |              |
  +------------------+ +----------------+ +----------------+ +--------------+
  Used for:            Used for:          Used for:
  - creating posts     - approving posts  - user management
  - editing directory  - rejecting posts  - audit log
  - resources/schedules- destructive ops  - site settings
```

⚠️ **Naming caveat:** `requireAdmin` *also lets managers in.* If you intend "no managers allowed," use `requireApprover`. This trips up new developers — the name does not match what the function does.

3.6 Frontend request lifecycle

```
1. Browser visits  http://<vm-ip>:8080/
2. nginx returns  frontend/dist/index.html
3. The HTML pulls /assets/index-xxxx.js  (the bundled React app)
4. React renders <App />
5. <SettingsContext> fires  fetch('/api/settings')  on mount
   -> nginx proxies to backend -> returns site-wide config
6. React Router resolves the URL to a <Page> component
7. Page mounts -> calls one of the helpers in src/services/api.js
   e.g.  getPosts()  ->  fetch('/api/posts', { credentials: 'include' })
8. Backend reads session cookie, returns approved posts only
9. React renders the data; user sees the page
```

All API calls go through `src/services/api.js`. It centralizes:

- The `/api` base path
- `credentials: 'include'` so the session cookie rides along
- Error normalization (throws `Error(message)` for non-2xx)

When adding a new API endpoint, put a wrapper here rather than calling `fetch` from a component.

3.7 Admin / manager portals

Intentionally **not** part of the React SPA:

- Plain HTML + vanilla JS — no build tool, no `npm install`
- Edits are immediately live; just refresh
- Zero dependencies — survive npm-ecosystem rot

Both portals call the same `/api/*` endpoints as the SPA. Mounted read-only into nginx at `/admin/` and `/manager/`.

3.8 REST API surface (high-level)

A non-exhaustive list — see `server.js` for complete signatures.

```
PUBLIC
GET    /api/health                        liveness
POST   /api/auth/login                    { username, password }
POST   /api/auth/logout
GET    /api/auth/me                       current user info
GET    /api/posts                         approved posts only
GET    /api/directory                     employee directory
GET    /api/resources, /schedules, /settings, /spotlight, /search
POST   /api/it-request                    open IT ticket
POST   /api/spotlight/nominations         submit nomination
GET    /api/signup-sheets[/:id]           list/view sign-up sheets

REQUIRE LOGIN (manager / admin / superadmin)
POST   /api/posts                         create new (status=pending)
POST   /api/resources                     upload resource
POST   /api/schedules                     create schedule

REQUIRE APPROVER (admin / superadmin)
GET    /api/admin/posts/pending           moderation queue
POST   /api/admin/posts/:id/approve
POST   /api/admin/posts/:id/reject

REQUIRE SUPERADMIN
GET    /api/admin/stats / audit / users
POST   /api/admin/users                   create user
PATCH  /api/admin/users/:id               update role/password
```

3.9 Rate limiting and security (current)

- **Login** endpoint is rate-limited to **20 attempts per 15 minutes** per IP.
- Passwords are hashed with **bcrypt** before storage.
- Sessions are HTTP-only cookies (`httpOnly: true` , `sameSite: 'lax'`).
- `secure: false` is set because nginx serves over plain HTTP today.

- nginx forwards `X-Real-IP` to the backend for accurate per-IP rate limiting.

⚠️ **Today's deployment is plain HTTP on a trusted LAN/Tailscale only.** Credentials cross the network in cleartext. This is acceptable in the homelab but **must be replaced with TLS before any production exposure** (see the migration plan below).

3.10 Email

`nodemailer` is configured with `jsonTransport` — emails are logged to stdout instead of being delivered. The intended Exchange SMTP block is commented out at the top of `server.js`. To enable real email, fill in the SMTP credentials there and `docker compose restart backend`.

3.11 State that lives outside the container

Host path	Container path	Why
`/srv/intranet/app/`	`/app/`	source code (live-mounted)
`/srv/intranet/data/`	`/data/`	SQLite databases
`/srv/intranet/uploads/`	`/uploads/`	uploaded files
`/srv/intranet/frontend/dist/`	`/usr/share/nginx/html/` (RO)	built SPA
`/srv/intranet/public/admin/`	`/admin/` (RO)	static admin portal
`/srv/intranet/public/manager/`	`/manager/` (RO)	static manager portal
`/srv/intranet/nginx/default.conf`	`/etc/nginx/conf.d/default.conf` (RO)	proxy config

3.12 🔄 Production migration

The application code is the largest single change between homelab and production. Planned changes:

3.12.1 Backend rewrite

Today (Express)	Target (Fastify)
`express`	`fastify`
`express-session` + `connect-sqlite3`	`@fastify/session` + `@fastify/cookie`
`multer` (multipart uploads)	`@fastify/multipart`
`express-rate-limit`	`@fastify/rate-limit`
`cors`	`@fastify/cors`
`better-sqlite3` (raw SQL)	`@prisma/client` (type-safe ORM)
Manual middleware functions	Fastify plugins + hooks
`app.get('/posts', ...)` route style	`fastify.get('/posts', { schema }, ...)` with JSON-schema validation

The route handlers and business logic translate one-to-one. The schema validation added by Fastify catches malformed payloads earlier; the type-safe Prisma queries replace handwritten SQL.

3.12.2 Database migration

Today (SQLite)	Target (PostgreSQL)
`/srv/intranet/data/intranet.db` (file)	`Intranet-HCI.heart.local:5432`, database `intranet_hci`
Schema in `db.exec()` inline	`prisma/schema.prisma` (typed)
No migrations	`prisma/migrations/` versioned
Inspect with `sqlite3`	Inspect with `psql` or DBeaver from any DBA workstation
Session store in `sessions.db`	Session store in the same Postgres DB
Backups: copy the `.db` file	Backups: `pg_dump -Fc`

3.12.3 TLS

Today	Target
Plain HTTP, port 8080	HTTPS on port 443, internal CA cert
`secure: false` on session cookie	`secure: true` on session cookie
No HSTS header	HSTS enabled in nginx

The migration is a single code branch — old and new cannot be partially deployed. Once the Fastify branch is approved through change management, the cutover replaces the entire backend container in one step. Data migration from SQLite to PostgreSQL is a one-time script that ships with the Fastify branch.

3.13 Where to go next

- `04-deployment.md` — Docker Compose mechanics
- `06-maintenance.md` — backup, rotate, and monitor the application

4 04 — Deployment

This document covers **how the intranet is packaged and run today** — Docker Compose, the two application containers, environment variables, and what happens when the system boots. The "Production migration" callout at the end lists the deployment changes planned for the Huntsville Hospital rollout.

4.1 Two-container topology (current)

`/srv/intranet/docker-compose.yml` defines two services:

```
services:
  web:
    image: nginx:1.30-alpine
    container_name: intranet_nginx
    ports:
      - "8080:80"
    volumes:
      - ./frontend/dist:/usr/share/nginx/html:ro
      - ./public/admin:/admin:ro
      - ./public/manager:/manager:ro
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - backend
    restart: unless-stopped

  backend:
    image: node:20-alpine
    container_name: intranet_backend
    working_dir: /app
    environment:
      - TZ=America/Chicago
      - NODE_ENV=production
      - SESSION_SECRET=${SESSION_SECRET}
    volumes:
      - ./app:/app
      - ./data:/data
      - ./uploads:/uploads
    command: sh -c "npm install && npm start"
    restart: unless-stopped
```

4.1.1 Key design choices

1. **No custom Dockerfiles.** Both services use stock public images (`nginx:1.30-alpine` and `node:20-alpine`). The application is supplied via bind-mounted source code.

2. **`npm install` runs on every container start.** Adding a dependency to `app/package.json` only requires `docker compose restart backend`.
3. **All persistent state is bind-mounted from the host.** No Docker named volumes. The host filesystem is canonical.
4. **`restart: unless-stopped`** — both services survive Docker daemon restarts and start automatically at boot.
5. **The web container `depends_on: backend`** so Compose starts the backend first. `depends_on` only waits for container start, not application readiness — see “Health and readiness” below.

4.2 Container lifecycle

```
$ docker compose up -d
                |
                v
+-------------------------------------------------------+
|  Pull images (if not cached)                          |
+-------------------------------------------------------+
                |
                v
+-------------------------------------------------------+
|  Create network 'intranet_default'  (172.18.0.0/16)   |
+-------------------------------------------------------+
                |
                v
+-------------------------------------------------------+
|  Start backend container                              |
|    - mount ./app, ./data, ./uploads                   |
|    - run npm install                                  |
|    - run npm start  ->  node server.js                |
|    - server binds 0.0.0.0:3000 inside container       |
+-------------------------------------------------------+
                |
                v
+-------------------------------------------------------+
|  Start web container                                  |
|    - mount frontend/dist, public/, nginx config       |
|    - nginx binds 0.0.0.0:80                           |
|    - Docker publishes host :8080  ->  container :80   |
+-------------------------------------------------------+
                |
                v
             READY
```

4.3 Networking inside Compose

```
intranet_nginx              intranet_backend
(service: web)              (service: backend)
    |  proxy_pass                 ^
    |   http://backend:3000/  -----+
    |
    v
host 0.0.0.0:8080  (exposed, HTTP)
```

The backend's port `3000` is **not** published to the host. It is reachable only from `web` (nginx).

4.4 Environment & secrets

One secret today: `SESSION_SECRET` in `/srv/intranet/.env`:

```
SESSION_SECRET=<hex string>
```

Compose reads `.env` automatically and substitutes `${SESSION_SECRET}` into the backend's environment.

The backend **refuses to start** if `SESSION_SECRET` is empty. Startup log:

```
FATAL: SESSION_SECRET environment variable is not set. Refusing to start.
```

To rotate (invalidates all sessions):

```
$ openssl rand -hex 48                                  # generate new
$ sudo nano /srv/intranet/.env                          # paste in
$ docker compose restart backend                        # apply
```

4.5 Day-to-day operations cheatsheet

Run from `/srv/intranet/`:

```
$ docker compose ps                 # show container state
$ docker compose up -d              # start (idempotent)
$ docker compose down               # stop and remove containers
$ docker compose restart            # restart both
$ docker compose restart backend    # just the backend
$ docker compose restart web        # just nginx
$ docker compose logs -f backend    # tail backend logs
$ docker compose logs -f web        # tail nginx logs
$ docker compose logs --tail=100    # last 100 lines of each
$ docker compose exec backend sh    # shell into backend container
$ docker compose exec web sh        # shell into nginx container
$ docker compose pull               # fetch newer image tags
```

Tip: `docker compose` is *declarative* and *idempotent*. Running `docker compose up -d` against an already-running stack is a no-op.

4.6 What “restart” actually does

```
   docker compose restart backend
       |
       +- send SIGTERM to the running container
       |    (10-second grace, then SIGKILL)
       |
       +- start a new container from the same image
       |
       +- re-mount the volumes
       |
       +- run the start command:
              sh -c "npm install && npm start"
              -- re-runs every restart, ~10-20 seconds
```

The backend takes ≈15–30 seconds to be ready after a restart because of `npm install`.

4.7 Building the frontend

The React app is **built ahead of time** and the artifacts in `frontend/dist/` are what nginx serves. **No hot-reload in the deployed stack.** After editing anything under `frontend/src/`:

```
$ cd /srv/intranet/frontend
$ npm run build           # produces fresh dist/  (10-30 s)
$ npm run lint            # optional, runs ESLint
```

⚠️ If you forget to rebuild, the website continues serving the old UI — this is the #1 confusing symptom for new contributors.

4.8 Editing the admin or manager portals

Plain HTML/JS files at:

```
/srv/intranet/public/admin/dashboard.html
/srv/intranet/public/admin/login.html
/srv/intranet/public/manager/index.html
```

Edits are **immediately live** on the next page load.

4.9 Health and readiness

No formal healthchecks in `docker-compose.yml`. The `GET /api/health` endpoint exists for manual probing:

```
$ curl -i http://localhost:8080/api/health
HTTP/1.1 200 OK
…
{"ok":true}
```

4.10 Cleanup recipes

```
$ docker compose down                # stop, keep data
$ docker system prune -f             # remove unused images / cache
$ sudo journalctl --vacuum-time=14d  # trim system logs
$ docker system df                   # Docker's disk footprint
```

4.11 🔄 Production migration

The deployment topology adds PostgreSQL and switches from HTTP:8080 to HTTPS:443. The Compose file gains an `extra_hosts` entry so the backend can reach the host's PostgreSQL through `host.docker.internal`.

4.11.1 Compose changes

```
# New ports section on the web service
ports:
  - "443:443"          # was "8080:80"
volumes:
  - ./nginx/tls:/etc/nginx/tls:ro    # NEW — mounts the TLS cert/key

# Backend changes
backend:
  extra_hosts:
    - "host.docker.internal:host-gateway"   # NEW — lets backend reach host
  environment:
    - DATABASE_URL=${DATABASE_URL}           # NEW — Postgres connection
    - INITIAL_ADMIN_PASSWORD=${INITIAL_ADMIN_PASSWORD}   # NEW
  command: sh -c "npm install && npx prisma migrate deploy && npm start"
```

4.11.2 One-time PostgreSQL setup on the production VM

```
$ sudo dnf install -y postgresql-server postgresql-contrib
$ sudo /usr/bin/postgresql-setup --initdb
$ sudo systemctl enable --now postgresql
# Configure listen_addresses and pg_hba.conf, then:
$ sudo systemctl reload postgresql
$ sudo -u postgres psql <<'SQL'
CREATE USER intranet_app WITH PASSWORD '...';
CREATE USER intranet_ro  WITH PASSWORD '...';
CREATE ROLE dba_group;
CREATE DATABASE intranet_hci OWNER intranet_app;
SQL
$ sudo firewall-cmd --permanent --add-port=5432/tcp
$ sudo firewall-cmd --reload
```

4.11.3 Updated `.env`

```
SESSION_SECRET=<48-byte hex>
DATABASE_URL=postgresql://intranet_app:<pw>@host.docker.internal:5432/intranet_hci?
schema=public
INITIAL_ADMIN_PASSWORD=<seeds admin on first boot>
```

4.11.4 Updated nginx config (TLS termination)

The container nginx config grows a `listen 443 ssl;` server block with `ssl_certificate` and `ssl_certificate_key` pointing at the bind-mounted cert from `/srv/intranet/nginx/tls/`.

4.11.5 Data migration

A one-time script (`scripts/migrate-sqlite-to-pg.js` to be written as part of the Fastify branch) reads from `intranet.db` and writes to PostgreSQL via Prisma. Run once on cutover day, after stopping the Express backend and before starting the Fastify backend.

4.12 Where to go next

- `05-remote-access.md` — getting into the box
- `06-maintenance.md` — routine care

5 05 — Remote Access

This document covers **all the ways into the server today** in the homelab environment — SSH, the desktop via RDP, the Cockpit web console, and the Tailscale overlay that secures remote access. Production access (Huntsville Hospital) drops Tailscale and RDP; see the migration callout at the end.

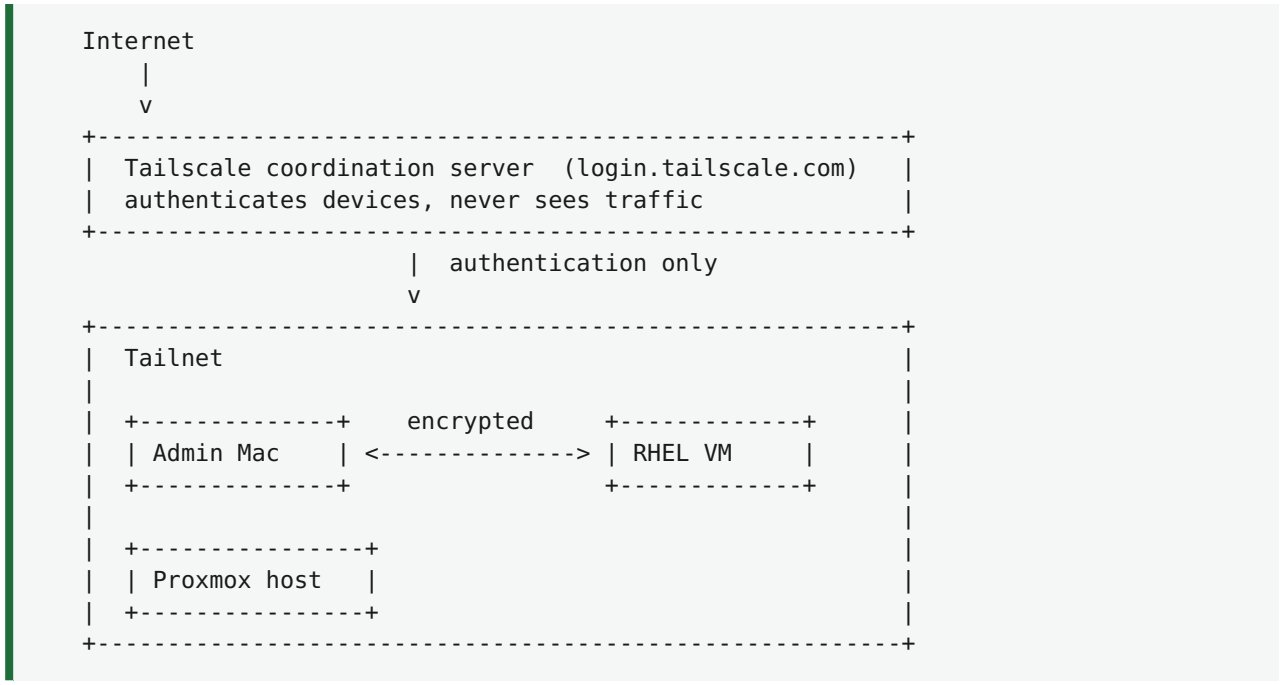
5.1 Access matrix (current homelab)

```
                +-------------------------------------------------------+
                |                   RHEL 10 VM                          |
                |                                                       |
                |   Port   Method            Purpose                    |
                |   ----   ----------------  ---------------------------- |
any device      |   22     SSH (OpenSSH)     Shell, file transfer, scripts |
----->------>|  8080    HTTP (nginx)      The intranet itself          |
                |  3389    RDP (g-r-d)       Full GNOME desktop, headless  |
                |  9090    HTTPS (Cockpit)   Web-based admin console      |
                |                                                       |
                |   *g-r-d = gnome-remote-desktop                       |
                +-------------------------------------------------------+

All four ports are reachable on BOTH the LAN address AND the Tailscale
address. SQLite-style DB inspection is via `docker compose exec backend`.
```

5.2 Tailscale (the secure overlay, homelab only)

Tailscale is a peer-to-peer encrypted mesh VPN that gives each authorized device a `100.x.y.z` address. Used here so the homelab VM can be reached from the admin's Mac without exposing any ports publicly.



5.2.1 Why we use it (homelab)

- No port-forwarding required on the homelab router
- End-to-end encrypted (WireGuard)
- Identity-based access — devices are authorized in the Tailscale admin console
- DNS works — devices can be reached by hostname (MagicDNS)

5.2.2 On the server: check Tailscale status

```
$ tailscale status              # list peers + self IP
$ tailscale ip                  # just our IPs
$ sudo systemctl status tailscaled
```

If Tailscale fails to come up:

```
$ sudo systemctl restart tailscaled
$ sudo tailscale up               # may prompt for login URL
```

5.3 SSH access

```
$ ssh <user>@<tailscale-ip>        # via Tailscale (anywhere)
$ ssh <user>@<lan-ip>              # via LAN (in the homelab)
```

Password: the local Linux password.

5.3.1 Recommended: switch to key-based SSH

```
$ ssh-keygen -t ed25519                                  # on your Mac
$ ssh-copy-id <user>@<tailscale-ip>                      # installs your pubkey
$ ssh <user>@<tailscale-ip>                              # no password needed
```

5.4 RDP — full GNOME desktop

The server runs **gnome-remote-desktop in headless (system) mode**. Every RDP connection spawns a fresh GNOME session.

5.4.1 Connection details

Setting	Value
Host	`<tailscale-ip>` or `<lan-ip>`
Port	`3389`
Username	local Linux user
Password	local Linux password
Security	Ignore self-signed certificate warnings

5.4.2 Recommended client: Royal TSX (free, native macOS)

Compatibility findings during homelab setup:

Client	Works?	Notes
Royal TSX (royalapps.com)	✅	Native, free, handles RDP server redirection
Microsoft “Windows App” (App Store)	⚠️	Cannot handle redirection used in headless mode
Jump Desktop	❌	NLA / NTLM incompatibility with gnome-remote-desktop
xfreerdp via Homebrew	⚠️	Renders through XQuartz — looks rough
virt-viewer + SPICE	❌	Bypassed in favor of RDP

To toggle full-screen in Royal TSX: **⌥⌘F** (Connection Full Screen).

5.4.3 What “headless mode” means

```
Royal TSX connects to :3389
    |
    v
gnome-remote-desktop daemon authenticates (TLS + credentials)
    |
    v
Daemon spawns a fresh GDM session for the configured user
    |
    v
Mutter creates a virtual monitor at the resolution Royal TSX requested
    |
    v
You see the desktop. Settings persist (same ~/.config), display layout doesn't.
```

When you disconnect, the GNOME session is torn down. Next connect = new session.

5.5 Cockpit (web console)

The friendliest way to manage the server:

```
https://<tailscale-ip>:9090/      <- via Tailscale
https://<lan-ip>:9090/            <- via LAN
```

Self-signed certificate — accept the warning. Log in with the local Linux user.

From Cockpit:

```
+-------------------------------------------------------------+
|  Cockpit (web)                                              |
|                                                             |
|  Overview          live CPU / memory / disk / network graphs  |
|  Logs              browse journald with search and filters    |
|  Storage           disk partitions, free space                |
|  Networking        interfaces, firewall, VPN                  |
|  Podman / Docker   running containers, restart, view logs     |
|  Software updates  list and install dnf updates with a UI     |
|  Services          systemd unit list, start/stop/enable       |
|  Accounts          list/create users, change passwords        |
|  Terminal          full shell access in the browser           |
+-------------------------------------------------------------+
```

For an operator who is not a Linux expert, **Cockpit is the single best tool to learn.** Most maintenance tasks in `06-maintenance.md` can be performed through Cockpit.

5.6 Local console (Proxmox)

If all remote paths fail, get into the VM via the Proxmox web UI:

1. Browse to the Proxmox host
2. Log in to Proxmox
3. Select VM **101** in the left tree
4. Click **Console** in the top right (use noVNC; SPICE is broken)

5.7 Quick reference: which tool when

You want to…	Use
Run a one-line command	SSH
Edit a file in vim/nano	SSH
Restart a container	SSH or Cockpit
Watch system metrics	Cockpit
Read logs with a search box	Cockpit
Use a graphical app (Firefox, Files)	RDP
Tweak GNOME settings	RDP
Update Red Hat packages	Cockpit or `sudo dnf upgrade`
Recover from boot problems	Proxmox noVNC console
Inspect the database	`docker compose exec backend` + `sqlite3`

5.8 🔄 Production migration

Remote access changes substantially when moving to Huntsville Hospital:

Item	Homelab (today)	Production target
Network reach	Tailscale + LAN	Hospital corporate network only
Tailscale	Active, all four ports reachable through it	Not installed
RDP via gnome-remote-desktop	On port 3389	Removed entirely; Cockpit-only
Web port	HTTP :8080	HTTPS :443
Cockpit	Self-signed cert, open from any LAN device	Hospital-issued cert, source-restricted to IT-Ops jump host
SSH	Password OR key	Key-only, source-restricted to IT-Ops jump host
Local console	Proxmox noVNC	vSphere Client (vCenter) → Open Console
New: DBA access	(n/a — SQLite inside container)	`psql` / DBeaver / pgAdmin direct to `Intranet-HCI.heart.local:5432`

5.8.1 What DBAs gain in production

Once PostgreSQL is in place, DBAs connect directly from their workstations without any container access. This is a major upgrade over today's `docker compose exec backend sqlite3` workflow, which requires shell access to the application VM.

5.8.2 DBA connection details (target)

Field	Value
Host	`Intranet-HCI.heart.local`
Port	`5432`
Database	`intranet_hci`
Authentication	as configured (LDAP / password / certificate, per hospital policy)
SSL mode	`require`

5.8.3 What DBAs can do (target)

- Run ad-hoc `SELECT` queries against any table
- Build reports against the read-only `intranet_ro` user
- Take logical backups with `pg_dump`

- Inspect query plans with `EXPLAIN ANALYZE`
- Tune indexes and statistics

5.8.4 What DBAs should NOT do (target, without coordination)

- Run `ALTER TABLE` or other schema changes — the application uses **Prisma migrations**; manual changes will drift from the codebase
- Drop or truncate tables on the live DB
- Change column types or constraints without a corresponding code change
- Run long blocking transactions during business hours

Schema changes flow through the codebase:

```
Developer edits prisma/schema.prisma
    |
    v
npx prisma migrate dev --name <change>
    |
    v
Commit migration file + schema.prisma
    |
    v
Code review + change-management approval
    |
    v
Production deploy applies via `prisma migrate deploy`
```

5.9 Where to go next

- `06-maintenance.md` — routine maintenance procedures
- `07-troubleshooting.md` — when something is broken

6 06 — Maintenance Guide

> ***This chapter is written for people new to Red Hat Linux.*** *It explains every command. If you are an experienced sysadmin, skim the code blocks; the prose is for newcomers. The procedures here describe the* ***current homelab*** *system; a "Production migration" callout at the end notes which procedures change for Huntsville Hospital.*

If you remember nothing else: **most problems are solved by restarting the right service.** Identify which layer is broken (web, backend, RDP, database, OS), then restart just that piece.

6.1 1. Orientation — what you're looking at

When you log in via SSH, you are dropped into a **shell** (the program that reads commands and runs them). The default shell here is `bash`. The prompt looks like:

```
<user>@<host>:~$
   |     |     | +- "I'm waiting for a command"
   |     |     +--- you are in your home folder (~/)
   |     +--------- the machine's hostname
   +--------------- your username
```

The most useful keys at the prompt:

Key	What it does
`Tab`	autocompletes file names and commands
`↑ / ↓`	cycles through command history
`Ctrl+R`	searches command history
`Ctrl+C`	cancels the currently running command
`Ctrl+L`	clears the screen
`Ctrl+D`	logs out

6.1.1 `sudo` — running things as root

Prefix any system-changing command with `sudo`. The first time per session it asks for your password. Subsequent `sudo` calls in the same shell skip the prompt for a few minutes.

```
$ whoami                    # -> <your user>
$ sudo whoami               # -> root  (after entering password)
```

⚠️ Always read the command before pressing Enter.

6.2 2. Daily / weekly checks

6.2.1 Is the website up?

```
$ curl -sI http://localhost:8080/        # should print "HTTP/1.1 200 OK"
$ curl -s http://localhost:8080/api/health
{"ok":true}
```

If either fails, jump to `07-troubleshooting.md` → *Website is down*.

6.2.2 Are the containers running?

```
$ cd /srv/intranet
$ docker compose ps
```

You should see both `intranet_nginx` and `intranet_backend` with status `Up`. Anything other than `Up` is a problem.

6.2.3 How is disk space?

```
$ df -h /                               # root filesystem free space
$ du -sh /srv/intranet/uploads/         # how much uploads have grown
$ du -sh /srv/intranet/data/            # SQLite database size
$ docker system df                      # Docker's view of disk usage
```

Treat anything above 80% used as urgent.

6.2.4 How is memory?

```
$ free -h
```

Expect 3–4 GiB used. If `available` drops below 500 MiB, something is leaking. Find the culprit:

```
$ top                  # 'q' to quit, 'M' to sort by memory, 'P' by CPU
```

💡 In Cockpit, the **Overview** page shows all of this graphically.

6.3 3. Restarting things (the most common operation)

```
   Restart NGINX only         (5 sec downtime)
       v
   Restart Backend only       (15-30 sec downtime)
       v
   Restart Both               (full intranet downtime, ~30 sec)
       v
   Restart Docker daemon      (~1 minute, restarts ALL containers)
       v
   Reboot the VM              (3-5 minutes total)
```

Run all of these from `/srv/intranet/`:

```
$ cd /srv/intranet
$ docker compose restart web              # just nginx
$ docker compose restart backend          # just the Node app
$ docker compose restart                  # both
$ sudo systemctl restart docker           # whole Docker daemon (RARE)
$ sudo systemctl reboot                   # full VM reboot
```

6.4 4. Reading logs

6.4.1 Application logs (the containers)

```
$ docker compose logs --tail=100 backend       # last 100 lines
$ docker compose logs --tail=100 web           # nginx
$ docker compose logs -f backend               # live tail (Ctrl+C to stop)
$ docker compose logs --since 1h backend       # past hour
```

6.4.2 System logs (anything outside the containers)

`journald` is the unified log service:

```
$ sudo journalctl -xe                    # last entries, formatted
$ sudo journalctl --since "1 hour ago"   # past hour
$ sudo journalctl --since today          # since midnight
$ sudo journalctl -u docker.service      # specific service
$ sudo journalctl -u sshd                # SSH
$ sudo journalctl -u gnome-remote-desktop  # RDP server
$ sudo journalctl -k                     # kernel only
$ sudo journalctl -f                     # live tail
```

In Cockpit, **Logs** in the left nav gives a search-and-filter UI.

6.5 5. Updating the operating system

```
$ sudo subscription-manager status        # registered?
$ sudo dnf check-update                   # list available updates
$ sudo dnf upgrade                        # apply (asks y/n)
$ sudo dnf upgrade -y                     # without prompt
```

After installing updates, check if a reboot is needed:

```
$ sudo dnf needs-restarting -r            # exits 1 if reboot recommended
```

If needed:

```
$ sudo systemctl reboot
```

6.5.1 Recommended update cadence

Cadence	Action
Weekly	`sudo dnf check-update`
Monthly	`sudo dnf upgrade --security`
Quarterly	Full `sudo dnf upgrade` + reboot

⚠️ Never run `dnf upgrade` mid-workday.

6.6 6. Backups (current — SQLite era)

6.6.1 What needs backing up

Importance	Path	Why
🔴 Critical	`/srv/intranet/data/`	Live SQLite databases
🔴 Critical	`/srv/intranet/uploads/`	Uploaded files
🟡 Important	`/srv/intranet/.env`	`SESSION_SECRET`
🟢 Nice	`/srv/intranet/app/` , `/srv/intranet/frontend/`	Source (also in git)

6.6.2 Manual backup recipe

```
$ DATE=$(date +%Y%m%d-%H%M%S)

# Safe SQLite snapshot (uses SQLite's online backup API)
$ docker compose exec backend sh -c \
    "apk add --no-cache sqlite >/dev/null 2>&1; \
     sqlite3 /data/intranet.db \".backup '/data/intranet-snapshot.db'\""

$ sudo tar -czf /home/<user>/intranet-backup-$DATE.tar.gz \
      -C /srv intranet/data intranet/uploads intranet/.env

$ sudo rm /srv/intranet/data/intranet-snapshot.db
```

Then copy off the VM:

```
# From your Mac:
$ scp <user>@<tailscale-ip>:/home/<user>/intranet-backup-*.tar.gz \
      ~/Backups/
```

6.6.3 Automated backups (recommended)

Save as `/usr/local/bin/intranet-backup.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
BACKUP_DIR=/var/backups/intranet
DATE=$(date +%Y%m%d-%H%M%S)
mkdir -p "$BACKUP_DIR"

# Online SQLite snapshot
docker compose -f /srv/intranet/docker-compose.yml exec -T backend sh -c \
  "sqlite3 /data/intranet.db \".backup '/data/intranet-snapshot.db'\""

tar -czf "$BACKUP_DIR/intranet-$DATE.tar.gz" \
    -C /srv intranet/data/intranet-snapshot.db intranet/uploads intranet/.env

rm /srv/intranet/data/intranet-snapshot.db

# Keep only the last 14 backups
ls -1t "$BACKUP_DIR"/intranet-*.tar.gz | tail -n +15 | xargs -r rm --
```

Schedule via cron:

```
$ sudo crontab -e
# Add:
0 2 * * * /usr/local/bin/intranet-backup.sh >> /var/log/intranet-backup.log 2>&1
```

⚠️ **Still copy them off the VM.** Sync to a NAS or cloud bucket weekly.

6.7 7. Disk space management

```
$ df -h /
$ sudo du -sh /var/* 2>/dev/null | sort -h        # biggest /var/ subdirs
$ docker system df                                # Docker footprint
$ sudo journalctl --disk-usage                    # log size
```

Cleanup commands:

```
$ docker system prune -a                          # ⚠ removes unused images
$ docker container prune                          # stopped containers
$ sudo journalctl --vacuum-time=14d               # trim journald
$ sudo dnf clean all                              # dnf cache
```

6.8 8. Managing users

```
$ sudo useradd -m -G wheel,docker alice           # creates home, gives sudo+docker
$ sudo passwd alice
$ sudo passwd -e alice                            # force password change on login
$ sudo userdel -r alice                           # -r removes home
```

⚠ **Application users** (admin accounts inside the intranet) are different — managed through the website's admin panel at `/admin/`, not via Linux.

6.9 9. Changing the intranet admin password (current — SQLite)

```
# 1. Generate a new bcrypt hash inside the backend container
$ docker compose exec backend node -e \
  "console.log(require('bcryptjs').hashSync('NewPass123!', 10))"
# Copy the resulting $2a$10$... string

# 2. Update the user row in SQLite
$ docker compose exec backend sh -c \
    "sqlite3 /data/intranet.db \
       \"UPDATE users SET password_hash='<paste-hash-here>' WHERE username='admin';
\""
```

6.10 10. Routine maintenance schedule

```
DAILY    (5 min, automated)
+- Backup script at 02:00
+- (Optional) cron health-check on /api/health

WEEKLY   (10 min, manual)
+- docker compose ps                  -> containers all Up
+- df -h /                            -> disk under 80%
+- sudo journalctl -p err --since=7d  -> any errors?
+- Tailscale admin console            -> expected devices online

MONTHLY  (30 min, manual, maintenance window)
+- sudo dnf upgrade --security
+- Review backup integrity (extract one, spot-check)

QUARTERLY (1 hour, maintenance window)
+- Full sudo dnf upgrade + reboot
+- Restart all containers from cold
+- Verify backups by TEST RESTORE
+- Review user list, deactivate inactive accounts
```

6.11 11. When in doubt

Three commands that have saved many sessions:

```
$ docker compose restart       # nudge the app
$ sudo systemctl reboot                          # nudge the OS
$ sudo journalctl -xe                            # what just broke?
```

If those don't help, see `07-troubleshooting.md`.

6.12 🔄 Production migration

When the system moves to Huntsville Hospital, several maintenance procedures change. The flowchart of "restart layers" stays the same; what changes is **how the database is backed up and inspected**, and that the **RDP layer is removed**.

6.12.1 Backups change from SQLite to PostgreSQL

Today	Target
`sqlite3 .backup` against a file in `data/`	`sudo -u postgres pg_dump -Fc intranet_hci`
Tar bundles the `.db` file	Tar bundles the `.dump` file
Restore: copy the file back	Restore: `pg_restore` into a fresh database

Sample target backup script:

```
#!/usr/bin/env bash
set -euo pipefail
DATE=$(date +%Y%m%d-%H%M%S)
sudo -u postgres pg_dump -Fc intranet_hci > /var/backups/intranet/intranet-db-$DATE.d
ump
tar -czf /var/backups/intranet/intranet-uploads-$DATE.tar.gz -C /srv intranet/uploads
```

6.12.2 Admin password reset (target) uses psql instead of sqlite3

```
$ docker compose exec backend node -e \
  "console.log(require('bcryptjs').hashSync('NewPass123!', 10))"
$ sudo -u postgres psql intranet_hci -c \
    "UPDATE \"User\" SET password_hash='<paste-hash-here>' WHERE username='admin';"
```

6.12.3 Update cadence becomes change-management driven

The hospital change-management process replaces the casual “monthly / quarterly” cadence. Each update window is documented in advance with a roll-back plan and a maintenance-window announcement.

6.12.4 Removed in production

- `gnome-remote-desktop` and GDM are uninstalled in production. RDP is no longer a maintenance path.
- Tailscale is uninstalled. Remote access is over the corporate network only.

6.12.5 Added in production

- **TLS certificate renewal** every 12 months. Coordinate with hospital PKI 30 days before expiry. Cert files at `/etc/nginx/tls/`.
- **PostgreSQL vacuum / analyze** weekly (cron): `sudo -u postgres vacuumdb --all --analyze`
- **DBA-driven schema migrations** via Prisma, applied automatically on backend restart (`prisma migrate deploy`).

7 07 — Troubleshooting

This is a **playbook of known problems and their fixes** for the current homelab deployment. When something is broken, find the matching symptom and follow the steps. Issues marked **(target)** apply to the production PostgreSQL migration, not the current SQLite system.

7.1 How to use this document

```
1. Identify the layer that's broken:
   • Can you SSH in?                          -> OS is up
   • Does the website respond at all?         -> nginx is up
   • Does /api/health work?                   -> backend is up
   • Do API calls return data?                -> database is reachable

2. Look for matching symptom below

3. Follow the playbook
```

7.2 Symptom index

7.3 Website is completely down

Symptom: Browser cannot reach `http://<vm-ip>:8080/`. Connection refused, or times out.

Triage:

```
$ ssh <user>@<tailscale-ip>         # can you log in?
$ ping <lan-ip>                     # is the VM on the network?
```

If you cannot ping or SSH, the VM is down or off the network. Check Proxmox.

If you can SSH, continue:

```
$ cd /srv/intranet
$ docker compose ps                 # are containers Up?
$ sudo ss -tlnp | grep 8080         # is port 8080 listening?
$ curl -I http://localhost:8080/    # local probe
```

Common causes and fixes:

1. **Containers stopped** — `docker compose up -d`
2. **Backend in restart loop** — `docker compose logs --tail=200 backend` reveals why. Usually `SESSION_SECRET` missing or a syntax error in `server.js`.
3. **nginx misconfig** — `docker compose logs --tail=50 web` shows nginx errors. "Host not found in upstream" means backend died.
4. **Firewall blocking port 8080**:

   ```
   $ sudo firewall-cmd --permanent --add-port=8080/tcp
   $ sudo firewall-cmd --reload
   ```

5. **Disk full** — `df -h /`. See Disk is full.

7.4 Website loads but shows error / blank page

Triage:

```
$ curl -i http://localhost:8080/api/health     # backend alive?
$ docker compose logs --tail=100 backend
```

HTTP 502 Bad Gateway: nginx can reach the backend but the backend isn't responding. Restart it:

```
$ docker compose restart backend
$ docker compose logs -f backend
```

HTTP 504 Gateway Timeout: backend is slow. Check load:

```
$ top                                  # is something at 100% CPU?
$ docker stats                         # container-level CPU/memory
```

Blank page: the React build is stale. Rebuild:

```
$ cd /srv/intranet/frontend
$ npm run build
```

7.5 API calls return 401 / “Not logged in”

Cause: Session cookie expired or `SESSION_SECRET` changed.

Fix: Log out and back in. If that fails for everyone, check the backend started with the secret:

```
$ docker compose logs backend | grep -i secret
# Should NOT see "FATAL: SESSION_SECRET environment variable is not set"
```

7.6 Login keeps failing

Possible causes:

1. **Rate limited.** 20 failed attempts in 15 minutes blocks the IP. Restart the backend to clear:

   ```
   $ docker compose restart backend
   ```

2. **Wrong password.** Reset — see I forgot the admin password.

3. **Backend can’t write to sessions.db.** Check disk and permissions:

   ```
   $ df -h /srv
   $ ls -la /srv/intranet/data/
   ```

7.7 Uploaded files don’t appear

Triage:

```
$ ls -la /srv/intranet/uploads/                    # is the file there?
$ docker compose exec backend ls -la /uploads/     # visible in container?
$ curl -I http://localhost:8080/files/<filename>
```

Common cause: the `/uploads` mount got disconnected after a Docker change. Recreate containers:

```
$ cd /srv/intranet
$ docker compose down
$ docker compose up -d
```

7.8 RDP connection hangs at “connecting”

Symptom: Royal TSX (or other client) shows “Connecting…” indefinitely.

Triage:

```
$ ss -tlnp | grep 3389                          # is it listening?
$ sudo systemctl status gnome-remote-desktop    # is the service up?
$ sudo journalctl -u gnome-remote-desktop -n 50 # recent logs
```

Common fix:

```
$ sudo systemctl restart gnome-remote-desktop
```

If credentials are empty in `grdctl --system status`:

```
$ sudo grdctl --system rdp set-credentials <user> '<password>'
$ sudo systemctl restart gnome-remote-desktop
```

If the TLS cert/key are gone:

```
$ sudo openssl req -new -x509 -days 3650 -nodes \
    -out /etc/gnome-remote-desktop/rdp-tls.crt \
    -keyout /etc/gnome-remote-desktop/rdp-tls.key \
    -subj "/CN=rhel-vm"
$ sudo grdctl --system rdp set-tls-cert /etc/gnome-remote-desktop/rdp-tls.crt
$ sudo grdctl --system rdp set-tls-key /etc/gnome-remote-desktop/rdp-tls.key
$ sudo systemctl restart gnome-remote-desktop
```

7.9 RDP error 0x207 (Windows App)

Symptom: Microsoft’s “Windows App” on Mac shows error `0x207` and disconnects immediately.

Cause: Headless gnome-remote-desktop uses **RDP server redirection**, which Microsoft’s Windows App on macOS does not honor.

Fix: Use a different client. Royal TSX (free) handles redirection correctly. See `05-remote-access.md` → *Recommended client*.

7.10 Tailscale is offline

Symptom: The VM doesn't appear in the Tailscale admin console, or `tailscale status` shows nothing.

Fix:

```
$ sudo systemctl restart tailscaled
$ sudo tailscale status                    # current state
$ sudo tailscale up                        # re-authenticate
                                           # will print a login URL
```

If `tailscale up` fails with "logged out":

```
$ sudo tailscale up --auth-key=tskey-xxxxx
```

(generate the auth key in the Tailscale admin console)

7.11 SSH refuses connection

Symptom: `ssh: connect to host … port 22: Connection refused`

Most likely: sshd is not running. From the Proxmox console:

```
$ sudo systemctl status sshd
$ sudo systemctl start sshd
$ sudo systemctl enable sshd
```

If the problem is Tailscale-only: - Check Tailscale: `sudo systemctl status tailscaled` - Check firewall: `sudo firewall-cmd --list-ports` should include `22/tcp` via `services: ssh`

7.12 Disk is full

Symptom: Random operations fail with "No space left on device." `df -h /` shows above 95% used.

Quick relief (in order):

```
# 1. Old Docker images and build cache
$ docker system prune -a -f

# 2. journald logs over 14 days
$ sudo journalctl --vacuum-time=14d

# 3. DNF cache
$ sudo dnf clean all

# 4. Anything in /tmp
$ sudo du -sh /tmp/*
$ sudo rm -rf /tmp/<stale-thing>
```

Find the biggest directories anywhere:

```
$ sudo du -h --max-depth=2 / 2>/dev/null | sort -rh | head -30
```

7.13 Containers keep restarting

Symptom: `docker compose ps` shows `Restarting` repeatedly.

Triage:

```
$ docker compose logs --tail=200 <service>
```

Look for the first error after each restart:

- **Backend:** `SESSION_SECRET` missing or `intranet.db` corrupted
- **Backend:** a recent code change has a syntax error
- **nginx:** `host not found in upstream "backend"` -> backend isn't running
- **nginx:** typo in `nginx/default.conf` -> test with `docker compose exec web nginx -t`

7.14 The SQLite database is locked

Symptom: Backend logs show `SQLITE_BUSY: database is locked`.

Cause: Something else has the database open (often a manual `sqlite3` session left in a forgotten terminal).

Fix:

```
$ docker compose exec backend sh
/app # ps -ef | grep sqlite
# kill any stray sqlite3 processes
/app # exit
$ docker compose restart backend
```

If the lock persists, the SQLite **WAL journal** may be corrupted. Restore from the most recent backup (see `08-disaster-recovery.md`).

(target) In the PostgreSQL production deployment this symptom changes to "P1001 / P2024 / pool exhaustion" errors. Diagnose with `sudo systemctl status postgresql` and `pg_stat_activity` queries.

7.15 I forgot the admin password

The Linux admin password (the local user account):

1. From the Proxmox noVNC console, reboot the VM
2. At the GRUB menu, press `e` to edit the boot entry
3. Find the line starting with `linux …`, add `rd.break enforcing=0`
4. Ctrl-X to boot
5. At the `switch_root:/#` prompt:

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd
exit
exit
```

The application admin password (the `admin` user in the website): See `06-maintenance.md` → *Changing the intranet admin password*.

7.16 SELinux is blocking something

Symptom: A service "should work" but doesn't, and `journalctl` mentions "AVC denied".

Check:

```
$ getenforce                                # current mode
$ sudo ausearch -m AVC --start recent       # recent denials
```

Quick test: temporarily set permissive to confirm SELinux is the cause:

```
$ sudo setenforce 0                          # permissive — NOT persistent
# test the failing operation
$ sudo setenforce 1                          # back to enforcing
```

The **proper** fix is labeling files correctly or generating a custom policy with `audit2allow`.

7.17 GPG check failed during dnf upgrade

Cause: A repository pushed a package signed by a key the system doesn't trust yet.

Fix options:

1. Refresh keys:

```
$ sudo dnf clean all
$ sudo subscription-manager refresh
$ sudo dnf upgrade
```

2. If a third-party repo is the culprit (e.g., EPEL):

```
$ sudo dnf upgrade --disablerepo=epel
```

3. Install the new key:

```
$ sudo rpm --import https://path/to/the/repo/KEY
```

7.18 When all else fails

```
$ docker compose down && docker compose up -d
$ sudo systemctl reboot
$ sudo journalctl -xe --since "10 minutes ago"
```

If the system still won't come up, restore from backup (`08-disaster-recovery.md`).

7.19 🔄 Production migration

In production these specific symptoms change or disappear:

Today's symptom	Production equivalent
RDP connection hangs	(removed — RDP not installed)
RDP error 0x207	(removed)
Tailscale is offline	(removed — Tailscale not installed)
SQLite database locked	Replaced by: PostgreSQL pool exhaustion / P1001 / P2024
`firewall-cmd --add-port=8080/tcp`	`firewall-cmd --add-port=443/tcp` (already open)
Login at `http://<ip>:8080`	Login at `https://Intranet-HCI.heart.local`
Restore SQLite by copying `.db` file	Restore via `pg_restore --no-owner -d intranet_hci`

A separate playbook for PostgreSQL-specific issues (connection pool, WAL size, autovacuum) will be added to v2.0 of this document once production has been running long enough to surface real incidents.

8 08 — Disaster Recovery

This document covers **what to do when something is broken badly enough that routine maintenance can't fix it.** The procedures here describe recovery for the current SQLite-based homelab deployment; the production migration callout at the end describes the equivalent Postgres-based procedures.

8.1 Recovery scenarios (current)

```
Scenario                          First step
---------------------------------  ---------------------------------
SQLite DB corrupted               Restore intranet.db from backup
Files deleted from /uploads/      Restore uploads from backup
/srv/intranet/ wiped entirely     Full application restore
Whole VM unbootable               Boot from Proxmox snapshot
Proxmox host fails                Restore VM image to new host
```

8.2 Scenario 1 — SQLite database is corrupted

Symptom: Backend keeps crashing with SQLite errors, or `sqlite3 intranet.db ".tables"` returns "file is not a database."

Recovery steps:

```
# 1. Stop the backend so it stops writing
$ cd /srv/intranet
$ docker compose stop backend

# 2. Move the bad database aside (don't delete — useful for forensics)
$ sudo mv data/intranet.db data/intranet.db.broken-$(date +%Y%m%d)

# 3. Find your latest good backup
$ ls -lh /var/backups/intranet/intranet-*.tar.gz

# 4. Extract just the database snapshot from the backup
$ sudo tar -xzf /var/backups/intranet/intranet-YYYYMMDD-HHMMSS.tar.gz \
    -C /tmp intranet/data/intranet-snapshot.db
$ sudo cp /tmp/intranet/data/intranet-snapshot.db /srv/intranet/data/intranet.db
$ sudo chown <user>:<user> /srv/intranet/data/intranet.db

# 5. Restart and verify
$ docker compose start backend
$ docker compose logs -f backend
$ curl http://localhost:8080/api/health
```

⚠️ Restoring the DB rolls back to the backup time. Posts, signups, and nominations made after the backup are **lost**. Communicate to users.

8.3 Scenario 2 — Uploads directory missing or wiped

```
$ docker compose stop backend                          # avoid mid-write

# Extract uploads from the backup
$ sudo tar -xzf /var/backups/intranet/intranet-LATEST.tar.gz \
    -C / srv/intranet/uploads

# Verify
$ ls /srv/intranet/uploads/ | head
$ docker compose start backend
```

If only some files are missing, extract to a temp dir and copy in only what you need:

```
$ mkdir /tmp/restore
$ sudo tar -xzf /var/backups/intranet/intranet-LATEST.tar.gz -C /tmp/restore
$ sudo cp /tmp/restore/srv/intranet/uploads/<file> /srv/intranet/uploads/
$ sudo rm -rf /tmp/restore
```

8.4 Scenario 3 — Whole `/srv/intranet/` is gone

```
# 1. Reinstall the directory layout from backup
$ sudo mkdir -p /srv/intranet
$ sudo tar -xzf /home/<user>/intranet-full-backup.tar.gz -C /srv

# 2. If your backups only include data/ and uploads/, re-pull source
#    code from the internal git repo.

# 3. Make sure .env is present
$ ls -la /srv/intranet/.env
# If missing, generate a NEW session secret (logs everyone out):
$ openssl rand -hex 48 | sudo tee /srv/intranet/.env
$ sudo vi /srv/intranet/.env
# File should contain one line:  SESSION_SECRET=<the-hex-string>

# 4. Rebuild the frontend if dist/ is empty
$ cd /srv/intranet/frontend
$ npm install
$ npm run build

# 5. Bring it back up
$ cd /srv/intranet
$ docker compose up -d
$ docker compose logs -f
```

8.5 Scenario 4 — VM won't boot

This is where **Proxmox snapshots** save you. If snapshots exist:

1. Open the Proxmox web UI
2. Select VM **101** → **Snapshots** tab
3. Pick the most recent good snapshot → **Rollback**
4. The VM reverts

⚠️ Rolling back discards everything after the snapshot, including database rows and uploaded files. **Combine with a recent backup restore.**

8.5.1 If no snapshots exist

If the VM disk is intact but the OS won't boot:

1. From Proxmox, **boot a rescue ISO** (RHEL boot media → "Rescue installed system")
2. It mounts your existing root at `/mnt/sysimage`
3. `chroot /mnt/sysimage`
4. Diagnose: `journalctl -xb`, `dnf history`, check `/etc/fstab`
5. `exit`, reboot

8.6 Scenario 5 — Total Proxmox host loss

Off-site asset	Restore path
Tar backups on a NAS	Stand up a new VM, run Scenario 3
Proxmox vzdump of the whole VM	Restore `.vma` to a new Proxmox host
Only the SQLite + uploads	Build fresh RHEL VM, install Docker, place files, deploy
Nothing	Rebuild from scratch using this documentation as the guide

The lesson: **always have at least the data backups stored off the hypervisor.**

8.7 Recovery time objectives (informational)

Scenario	Estimated downtime
Database corruption only	5–10 minutes
Uploads only	10–30 minutes (depends on size)
`/srv/intranet/` wiped	30–60 minutes
VM unbootable, snapshot available	5–15 minutes
VM unbootable, no snapshot	1–3 hours
Total hypervisor loss	4–8 hours

8.8 Testing your recovery procedure

⚠️ **A backup you have never restored is not a backup.** Test quarterly:

```
# 1. Scratch directory
$ mkdir /tmp/restore-test

# 2. Extract latest backup
$ sudo tar -xzf /var/backups/intranet/intranet-LATEST.tar.gz -C /tmp/restore-test

# 3. Check the database is readable
$ sudo sqlite3 /tmp/restore-test/srv/intranet/data/intranet-snapshot.db \
    "SELECT COUNT(*) FROM users; SELECT COUNT(*) FROM posts;"

# 4. Spot-check uploads
$ ls /tmp/restore-test/srv/intranet/uploads/ | head
$ file /tmp/restore-test/srv/intranet/uploads/* | head

# 5. Clean up
$ sudo rm -rf /tmp/restore-test
```

If any step fails, your backups are not viable.

8.9 🔄 Production migration

Every scenario above translates to a Postgres equivalent in production:

8.9.1 Scenario 1 (database) — Postgres version

```
$ docker compose stop backend

# Take a forensic dump of the broken state
$ sudo -u postgres pg_dump -Fc intranet_hci > \
    /var/backups/intranet/intranet-BROKEN-$(date +%Y%m%d-%H%M%S).dump

# Drop and recreate
$ sudo -u postgres psql -c "DROP DATABASE intranet_hci;"
$ sudo -u postgres psql -c "CREATE DATABASE intranet_hci OWNER intranet_app;"
$ sudo -u postgres pg_restore --no-owner --role=intranet_app \
    -d intranet_hci /var/backups/intranet/intranet-db-LATEST.dump

$ docker compose start backend
```

8.9.2 Scenario 4 (boot) — vSphere version

In production, snapshots are taken in the vSphere Client: 1. Right-click VM `Intranet-HCI` → Snapshots → Manage Snapshots 2. Select a snapshot → **Revert**

OVA exports replace `vzdump` for migrating to another hypervisor.

8.9.3 New scenario in production: Postgres data dir corrupted

If `/var/lib/pgsql/data` is corrupted but the VM otherwise boots:

```
$ sudo systemctl stop postgresql
$ sudo mv /var/lib/pgsql/data /var/lib/pgsql/data.broken-$(date +%Y%m%d)
$ sudo /usr/bin/postgresql-setup --initdb
# Re-configure postgresql.conf and pg_hba.conf from configuration backup
$ sudo systemctl start postgresql
$ sudo -u postgres psql -c "CREATE DATABASE intranet_hci OWNER intranet_app;"
$ sudo -u postgres pg_restore --no-owner --role=intranet_app \
    -d intranet_hci /var/backups/intranet/intranet-db-LATEST.dump
$ cd /srv/intranet && docker compose restart backend
```

8.9.4 Updated recovery time objectives (target)

Scenario	Estimated downtime
Database corruption only	10–20 minutes
Postgres data dir corrupted	20–45 minutes
Uploads only	10–30 minutes
`/srv/intranet/` wiped	30–60 minutes
VM unbootable, snapshot available	5–15 minutes
VM unbootable, no snapshot	1–3 hours
Total vSphere host loss	4–8 hours

8.10 Building a runbook for your specific environment

Once a year, write a one-page “if I had to do this from scratch today” doc:

- Where do the backups live? (Path, hostname, credentials)
- Who is the Proxmox / vSphere admin contact?
- Who owns the Tailscale tailnet (homelab) or hospital corp network (production)?
- Where is the Red Hat subscription tied to?
- Who else knows? Designate a backup operator.

Store this off-site. Print a copy. The day you need it, you may not have internet access.

9 09 — Glossary

A reference for the terms used throughout this documentation. Skim it once, then refer back as needed. Some terms apply only to the **current homelab** deployment, others only to the **production target** — these are flagged where relevant.

9.1 Networking

LAN (Local Area Network) — the practice's internal network. Today the homelab LAN; in production, the Huntsville Hospital corporate VLAN.

Tailscale (homelab only) — a mesh VPN that gives every authorized device a `100.x.y.z` address. Used for remote admin access in the homelab. Removed in production.

Tailnet (homelab only) — a Tailscale network owned by one account.

Reverse proxy — a server that accepts requests on behalf of another. Here, nginx accepts HTTP (homelab) or HTTPS (production) requests and forwards them to the backend on port 3000.

Docker bridge network — a virtual switch that lets containers talk to each other by name. The intranet uses `intranet_default`.

Docker bridge gateway (target) — the host-side endpoint of the Docker bridge (typically `172.18.0.1`). In production the Fastify backend uses it to reach PostgreSQL on the host.

`host.docker.internal` (target) — hostname Docker can provide inside containers, resolving to the host. The backend's `DATABASE_URL` uses this.

Loopback — `127.0.0.1`, "this machine."

9.2 Operating system

RHEL — Red Hat Enterprise Linux. The OS this VM runs.

dnf — RHEL's package manager.

rpm — the low-level package format used by dnf.

systemd — RHEL's init system and service manager.

systemctl — the command to control systemd units.

Unit / service — a thing systemd manages.

journald / journalctl — systemd's log collector and query tool.

SELinux — Red Hat's mandatory access control system.

firewalld — RHEL's firewall manager.

SSH — Secure Shell. Encrypted remote-login protocol on port 22.

Cockpit — Red Hat's web-based system administration console (port 9090).

9.3 Application stack

SPA (Single Page Application) — a web application where one HTML page loads and JavaScript handles all subsequent navigation. The employee-facing React app is an SPA.

Express (homelab) — the current Node.js web framework. Single-file backend in `app/server.js`.

Fastify (target) — A fast, low-overhead web framework for Node.js. Replaces Express in production. Modular plugins replace middleware.

REST API — the HTTP-based interface exposed by the backend. Verbs: GET, POST, PATCH, DELETE.

Session — a server-side record that identifies a logged-in user. Identified by a cookie the browser sends with every request.

Session secret — a random string used to cryptographically sign session cookies. Stored in `/srv/intranet/.env` as `SESSION_SECRET`.

bcrypt — a password hashing algorithm. Slow on purpose, to resist brute-force attacks.

Middleware / Plugin — A piece of functionality that runs in the request pipeline. Express calls them middleware; Fastify calls them plugins. We use them for authentication checks, session handling, file uploads, rate-limiting.

ORM — an Object-Relational Mapper. The current homelab implementation does NOT use one; SQL is written directly via `better-sqlite3`. The production target uses **Prisma**, a type-safe ORM.

Prisma (target) — A modern Node.js ORM. Schema in `prisma/schema.prisma`, migrations under `prisma/migrations/`, generated client at `@prisma/client`.

Migration (target) — A versioned SQL file that brings the database schema from one state to the next. Applied automatically on backend startup via `prisma migrate deploy`.

9.4 Database

SQLite (homelab) — A file-based SQL database. The whole database is one `.db` file. No separate server process. Today's implementation uses `better-sqlite3` for synchronous in-process access.

PostgreSQL (target) — A mature, open-source relational database. Runs as a multi-process server on the host (not in a container). Listens on TCP port 5432.

psql (target) — PostgreSQL's command-line client.

pg_dump / pg_restore (target) — PostgreSQL's logical backup and restore tools. `-Fc` produces a compressed custom-format dump.

WAL (Write-Ahead Log) — A journal of pending changes. Both SQLite and PostgreSQL use it for crash recovery.

Role / User (PostgreSQL, target) — A database principal. The intranet plans to use `intranet_app`, `intranet_ro`, and `dba_group`.

pg_hba.conf (target) — PostgreSQL's host-based authentication file.

9.5 Containers and Docker

Container — a sandboxed process running on the host kernel.

Image — a snapshot of a filesystem used to start containers. We use stock public images.

Docker Compose — a tool that defines and runs multi-container apps from a single `docker-compose.yml` file.

Bind mount — exposing a host directory inside a container. The host filesystem is canonical.

Volume (named) — a Docker-managed storage area. We do not use these; only bind mounts.

Restart policy — Docker's instruction for what to do when a container stops. We use `unless-stopped`.

`extra_hosts` (target) — A Compose directive that injects entries into a container's `/etc/hosts`. Used in production to give the backend a way to reach the host as `host.docker.internal`.

9.6 Security and TLS

TLS (Transport Layer Security) — the protocol behind HTTPS.

Certificate — A cryptographic document binding a hostname to a public key. In production, issued by the Huntsville Hospital internal CA.

Private key — the secret half of a TLS certificate.

HSTS (HTTP Strict Transport Security) — A response header that tells browsers to use HTTPS only for a given host. Enabled in production.

CSRF (Cross-Site Request Forgery) — An attack class mitigated by the SameSite=Lax cookie attribute used for session cookies.

9.7 Virtualization

Proxmox VE (homelab) — The hypervisor today. KVM-based.

vSphere (target) — VMware's enterprise virtualization platform. Huntsville Hospital's production hypervisor environment.

ESXi (target) — The hypervisor itself.

vCenter (target) — The management interface for ESXi hosts.

Snapshot — A point-in-time copy of a VM. Both Proxmox and vSphere offer this; rolling back discards changes made after the snapshot.

OVA / OVF (target) — Portable VM image formats used for migrating between vSphere clusters.

9.8 Remote desktop (homelab only)

SPICE — A Linux remote-display protocol. We attempted this with Proxmox and abandoned it for RDP.

RDP — Microsoft's Remote Desktop Protocol. Used in the homelab via `gnome-remote-desktop` on port 3389.

gnome-remote-desktop — The GNOME project's RDP server. Used in headless mode here, which spawns a fresh GNOME session per connection.

Wayland — The modern Linux display server protocol. GNOME on RHEL 10 runs on Wayland by default.

Mutter — GNOME's window manager / compositor.

9.9 Storage

LVM (Logical Volume Manager) — A Linux abstraction over physical disks.

xfs — The default filesystem on RHEL 10.

inode — A filesystem object that holds metadata.

9.10 Frontend

React — A JavaScript library for building user interfaces. The employee-facing site is a React 19 application.

React Router — Routing for SPAs.

Tailwind — A utility-class CSS framework.

Vite — A modern frontend build tool.

JSX — JavaScript with embedded XML-like syntax.

9.11 Miscellaneous

dotfile — A file or directory whose name starts with `.`.

SIGTERM / SIGKILL — Process signals. SIGTERM asks a process to clean up and exit; SIGKILL terminates immediately.

9.12 Common abbreviations used in this document

- `cwd` — current working directory
- `pid` — process ID
- `uid / gid` — user / group ID
- `env` — environment (variables)
- `pkg` — package
- `db` — database
- `fs` — filesystem
- `ro / rw` — read-only / read-write
- `FQDN` — fully qualified domain name
- `DBA` — database administrator
- `CA` — certificate authority
- `RDBMS` — relational database management system